



Πανεπιστήμιο Αιγαίου

Τμήμα Μηχανικών Πληροφοριακών και Επικοινωνιακών
Συστημάτων

Τεχνολογία Λογισμικού

Διδάσκων: Κρητικός Κυριάκος (Θεωρία), Φώτιος Κατσίφας (Εργαστήριο)

Απαλλακτική Εργασία : ανάπτυξη backend ενός
πληροφοριακού συστήματος για τη διαχείριση μουσικών φεστιβάλ
και των μουσικών
εμφανίσεων που υποβάλλονται σε αυτά.

321/2022246

Χουλιάρης Ιωάννης

Σάμος 13/01/2026

Πίνακας Περιεχομένων

Οργάνωση

- 1.1 Βασικοί ρόλοι στη σχεδίαση και ανάπτυξη και ανάθεση στην ομάδα
- 1.2 Βασικές δραστηριότητες/στάδια ανάπτυξης και διάρκειες

Απαιτήσεις

- 2.1 Λειτουργικές Απαιτήσεις
- 2.2 Μη Λειτουργικές Απαιτήσεις

Σχεδίαση

- 3.1 Διάγραμμα Συστατικών (Component Diagram)
- 3.2 Διάγραμμα Περιβάλλοντος (Context Diagram)
- 3.3 Διαγράμματα Περιπτώσεων Χρήσης (Use Case Diagrams)
- 3.4 Διαγράμματα Δραστηριοτήτων (Activity Diagrams)
- 3.5 Διαγράμματα Ακολουθίας (Sequence Diagrams)
- 3.6 Διάγραμμα Οντοτήτων-Συσχετίσεων (ER Diagram)

Υλοποίηση συστήματος/εφαρμογής

- 4.1 GitHub Repository και τεκμηρίωση λειτουργίας API endpoints
- 4.2 Υλοποίηση και σχολιασμός
- 4.3 Τεκμηρίωση δοκιμών

5. Συμπεράσματα

6. Πηγές

Οργάνωση

1.1 Βασικοί ρόλοι στη σχεδίαση και ανάπτυξη και ανάθεση στην ομάδα

Η ανάπτυξη του συστήματος πραγματοποιήθηκε στο πλαίσιο ατομικού project, με στόχο την κάλυψη ολόκληρου του κύκλου ζωής λογισμικού: απαιτήσεις, σχεδίαση, υλοποίηση και δοκιμές.

1.2 Βασικές δραστηριότητες/στάδια ανάπτυξης και διάρκειες

Η εργασία οργανώθηκε σε διακριτά στάδια που αντιστοιχούν στα τρία κύρια μέρη του έργου (Απαιτήσεις, Σχεδίαση, Υλοποίηση/Δοκιμές) και στη συγγραφή της τελικής αναφοράς. Η παρακολούθηση προόδου έγινε μέσω GitHub repository και σταδιακών commits.

Στάδια ανάπτυξης και χρόνος απασχόλησης

Στάδιο	Περιγραφή	Διάρκεια (ώρες)
Ανάλυση απαιτήσεων	λειτουργικές/μη λειτουργικές απαιτήσεις, παραδοχές, ορισμός ρόλων	4
Σχεδίαση	UML διαγράμματα (context/component/use case/activity/sequence) + ER	10
Υλοποίηση backend	endpoints, υπηρεσίες, persistence, security, validations	35
Υλοποίηση frontend	σελίδες/ροές, σύνδεση με API, role-aware UI	25

Στάδιο	Περιγραφή	Διάρκεια (ώρες)
Δοκιμές & διορθώσεις	unit tests, έλεγχοι ροών, bug fixes	40
Τεκμηρίωση	αναφορά, οδηγίες εκτέλεσης, screenshots, Insomnia requests	8
Σύνολο		122

Απαιτήσεις

Στο κεφάλαιο αυτό καταγράφονται οι απαιτήσεις του συστήματος “FestivalManager”, το οποίο υποστηρίζει τη διαχείριση φεστιβάλ/προγραμμάτων και την καταχώριση–αξιολόγηση–δημοσίευση παραστάσεων (performances). Η λειτουργικότητα οργανώνεται γύρω από ρόλους χρηστών (ενδεικτικά: Visitor/Authenticated User, Submitter/Artist, Staff/Programmer, Organizer) και από έναν σαφή κύκλο ζωής (καταστάσεις) μιας παράστασης, ώστε να εφαρμόζονται κανόνες πρόσβασης και ορθότητας σε κάθε βήμα.

Ορισμοί (σύντομα)

Παράσταση (Performance): η οντότητα που υποβάλλεται, αναθεωρείται και εγκρίνεται/απορρίπτεται.

Πρόγραμμα (Program/Festival Program): σύνολο παραστάσεων/εκδηλώσεων που ανήκουν σε συγκεκριμένο φεστιβάλ/έκδοση.

Κατάσταση παράστασης: διακριτή φάση στο workflow (π.χ. CREATED, SUBMITTED, UNDER_REVIEW, APPROVED/REJECTED, WITHDRAWN, SCHEDULED/ANNOUNCED, όπου εφαρμόζεται).

2.1 Λειτουργικές Απαιτήσεις

Οι λειτουργικές απαιτήσεις (FR) περιγράφουν τι πρέπει να κάνει το σύστημα. Για λόγους ιχνηλασιμότητας, κάθε απαίτηση έχει μοναδικό κωδικό και αντιστοιχίζεται έμμεσα με τα Use Case / Activity / Sequence Diagrams του Κεφαλαίου 3 και με τα endpoints του Κεφαλαίου 4.

Κωδικός	Περιγραφή απαίτησης	Κύριος ρόλος/Actor	Προτεραιότητα
FR-01	Το σύστημα επιτρέπει σύνδεση/αποσύνδεση χρήστη και αναγνωρίζει τον ρόλο του.	Authenticated User	Must
FR-02	Το σύστημα εφαρμόζει έλεγχο πρόσβασης (authorization) σε όλες τις ενέργειες βάσει ρόλων.	Όλοι	Must
FR-03	Το σύστημα επιτρέπει δημιουργία/ενημέρωση φεστιβάλ ή προγράμματος (βασικά μεταδεδομένα, χρονικά όρια όπου απαιτείται).	Organizer/Staff	Must
FR-04	Το σύστημα επιτρέπει δημιουργία παράστασης με έλεγχο εγκυρότητας (υποχρεωτικά πεδία, μοναδικότητα ονόματος όπου έχει οριστεί) και αποθήκευση.	Staff/Organizer ή Submitter (ανά πολιτική)	Must
FR-05	Κατά τη δημιουργία/υποβολή παράστασης, το σύστημα αποδίδει αυτόματα συσχετιζόμενους ρόλους όπου ισχύει (π.χ. ARTIST/SUBMITTER) και καταγράφει ιδιοκτησία/συσχέτιση.	Σύστημα	Should
FR-06	Το σύστημα επιτρέπει υποβολή (submit) παράστασης προς αξιολόγηση και αλλάζει την κατάστασή της σύμφωνα με το workflow.	Submitter/Artist	Must
FR-07	Το σύστημα επιτρέπει απόσυρση (withdraw) υποβληθείσας παράστασης, όταν αυτό επιτρέπεται από τους κανόνες κατάστασης.	Submitter/Artist	Should
FR-08	Το σύστημα επιτρέπει ανάθεση χειριστή/υπεύθυνου (handler) σε	Staff/Organizer	Should

Κωδικός	Περιγραφή απαίτησης	Κύριος ρόλος/Actor	Προτεραιότητα
	παράσταση από εξουσιοδοτημένο ρόλο.		
FR-09	Το σύστημα παρέχει αναζήτηση παραστάσεων με φίλτρα (προαιρετικά), με AND- σημασιολογία, καθώς και ταξινόμηση (sorting).	Όλοι (με ρόλο- κανόνες)	Must
FR-10	Το σύστημα παρέχει προβολή παράστασης ανά id, με απόκρυψη/περιορισμό πεδίων ανά ρόλο (role-aware redaction).	Visitor / User / Staff / Organizer	Must
FR-11	Το σύστημα υποστηρίζει αξιολόγηση παράστασης (review) και λήψη απόφασης (approve/reject) από εξουσιοδοτημένους ρόλους.	Staff/Organizer	Must
FR-12	Το σύστημα επιτρέπει ενημέρωση (update) παράστασης μόνο όταν πληρούνται οι κανόνες (ενδεικτικά: κατάσταση CREATED και δικαίωμα ιδιοκτήτη/ρόλου).	Owner + κατάλληλος ρόλος	Must
FR-13	Το σύστημα επιτρέπει οριστικοποίηση/ανακοίνωση προγράμματος (publishing/announcement) όπου αυτό προβλέπεται στο workflow.	Organizer	Could
FR-14	Το σύστημα παρέχει ιστορικό βασικών ενεργειών (ποιος/πότε) για ελεγκτικούς λόγους, τουλάχιστον για κρίσιμες μεταβάσεις (submit/review/approve/reject).	Staff/Organizer	Should

Σημείωση: Οι ακριβείς καταστάσεις και μεταβάσεις ορίζονται στο αντίστοιχο state/workflow της παράστασης και αποτυπώνονται στα Activity/Sequence Diagrams της Σχεδίασης.

Παρατηρήσεις:

FR-05 (auto-assign ρόλων): δεν υπάρχει αυτόματη ανάθεση ρόλου σε επίπεδο backend· οι ρόλοι δίνονται μέσω role assignment στο festival.

FR-07 (withdraw): υλοποιείται ως “withdraw μόνο σε CREATED” και πρακτικά κάνει delete. Δεν υποστηρίζει απόσυρση μετά το submit.

FR-13 (announce/publish): η “δημοσίευση” προσεγγίζεται μέσω SCHEDULED + DECISION workflow, αλλά δεν υπάρχει ξεχωριστό “publish program” endpoint όπως το γράψαμε.

FR-14 (audit): υπάρχουν timestamps και reviews, αλλά όχι πλήρες audit log για όλες τις ενέργειες.

2.2 Μη Λειτουργικές Απαιτήσεις

Οι μη λειτουργικές απαιτήσεις (NFR) περιγράφουν περιορισμούς και ποιοτικά χαρακτηριστικά του συστήματος (ασφάλεια, επίδοση, αξιοπιστία, συντηρησιμότητα κ.λπ.).

Κωδικός	Κατηγορία	Απαίτηση
NFR-01	Ασφάλεια	Όλες οι λειτουργίες που τροποποιούν δεδομένα απαιτούν αυθεντικοποίηση και έλεγχο εξουσιοδότησης βάσει ρόλων.
NFR-02	Ασφάλεια	Τα credentials δεν αποθηκεύονται σε απλή μορφή· εφαρμόζεται ασφαλής αποθήκευση/κρυπτογράφηση και πολιτική ισχυρών κωδικών όπου προβλέπεται.

Κωδικός	Κατηγορία	Απαίτηση
NFR-03	Ακεραιότητα δεδομένων	Το σύστημα επιβάλλει κανόνες εγκυρότητας (validations), περιορισμούς (π.χ. uniqueness όπου ορίζεται) και συνεπείς συσχετίσεις οντοτήτων.
NFR-04	Αξιοπιστία	Σε περίπτωση σφάλματος, το σύστημα επιστρέφει σαφή μηνύματα/κωδικούς HTTP και δεν αφήνει τα δεδομένα σε ενδιάμεση ασυνεπή κατάσταση (atomic updates).
NFR-05	Επίδοση	Η αναζήτηση παραστάσεων ολοκληρώνεται σε αποδεκτό χρόνο για τυπικό φόρτο (π.χ. χρήση pagination/ordering όπου απαιτείται).
NFR-06	Ευχρηστία	Το UI διατηρεί συνεπή πλοήγηση και εμφανίζει ενέργειες ανά ρόλο (role-aware UI), ώστε ο χρήστης να βλέπει μόνο τις διαθέσιμες επιλογές.
NFR-07	Συντηρησιμότητα	Ο κώδικας ακολουθεί διαστρωμάτωση (controllers/services/repositories), σαφή DTO boundaries και σχολιασμό σε κρίσιμα σημεία επιχειρησιακής λογικής.
NFR-08	Δοκιμασιμότητα	Παρέχονται unit/integration tests για βασικούς κανόνες (π.χ. status transitions, role checks, search filters) και επαναληψιμότητα εκτέλεσης.
NFR-09	Καταγραφή ενεργειών	Υπάρχει logging για κρίσιμα γεγονότα (login, submit, review, approve/reject, handler assignment) για υποστήριξη debugging/auditing.
NFR-10	Φορητότητα	Το σύστημα εκτελείται με τυπικά εργαλεία ανάπτυξης (backend build/run, frontend build/run) και τεκμηριώνεται η διαδικασία εκκίνησης.
NFR-11	Συμβατότητα	Η επικοινωνία γίνεται μέσω REST/JSON και το frontend στοχεύει σύγχρονους browsers.

Παρατηρήσεις:

NFR-09 (structured logging/auditing) : Δεν υλοποιήθηκε

Σχεδίαση

Στο κεφάλαιο της σχεδίασης παρουσιάζεται η αρχιτεκτονική δομή του συστήματος, η αλληλεπίδρασή του με εξωτερικούς χρήστες/εργαλεία, καθώς και η μοντελοποίηση βασικών λειτουργιών μέσω UML διαγραμμάτων. Η σχεδίαση ακολουθεί διαστρωμάτωση (presentation–service–persistence) και αποτυπώνει το ροϊκό μοντέλο καταστάσεων (workflow) που διέπει τις μεταβάσεις μιας παράστασης και τα αντίστοιχα δικαιώματα ανά ρόλο.

3.1 Διάγραμμα Συστατικών (Component Diagram)

Το Διάγραμμα Συστατικών αποτυπώνει τα βασικά υποσυστήματα του FestivalManager και τις διεπαφές επικοινωνίας μεταξύ τους. Η συνολική αρχιτεκτονική βασίζεται σε τρι-επίπεδη λογική:

(α) Frontend (React εφαρμογή):

Αποτελεί το επίπεδο παρουσίασης (UI) και υλοποιεί τις βασικές σελίδες/ροές (π.χ. Dashboard, Programs, Submissions, Screenings/Scheduling). Επικοινωνεί με το backend μέσω HTTP(S) κλήσεων REST και προσαρμόζει τις διαθέσιμες ενέργειες ανά ρόλο χρήστη (role-aware UI). Η αυθεντικοποίηση πραγματοποιείται με HTTP Basic headers, ώστε τα endpoints που απαιτούν authentication να καλούνται με τα κατάλληλα διαπιστευτήρια.

(β) Backend (Spring Boot REST API):

Το backend εκθέτει REST endpoints (π.χ. /api/festivals, /api/performances, /api/auth/me) και συγκεντρώνει την επιχειρησιακή λογική στους service classes. Στο επίπεδο αυτό εφαρμόζονται οι κανόνες:

- επιτρεπτές μεταβάσεις κατάστασης (festival state / performance state),
- έλεγχοι ρόλων (PROGRAMMER/STAFF κ.λπ.) σε κρίσιμες ενέργειες (assignment, review, decision),
- “role-aware visibility” στις προβολές τύπου search-view (τι βλέπει ο visitor vs staff/handler vs programmer).

(γ) Persistence (JPA/Hibernate + H2):

Το επίπεδο δεδομένων υλοποιείται μέσω JPA repositories και οντοτήτων (Festival, Performance, Review, User, Role, UserFestivalRole). Η βάση (H2 στο dev profile) τροφοδοτείται με αρχικά δεδομένα όπου απαιτείται, ώστε να δοκιμάζονται σενάρια ρόλων και ροών. Οι συσχετίσεις (many-to-one, one-to-

many) υποστηρίζουν την αποθήκευση των reviews, την ανάθεση handler, και τις αναζητήσεις ανά festivalId.

Εξωτερικές οντότητες/εργαλεία (εκτός συστήματος):

Χρήστης μέσω browser (πλοήγηση UI) και εργαλείο API testing (Insomnia) για δοκιμή endpoints/σεναρίων. Τα στοιχεία αυτά εμφανίζονται στο component diagram ως “clients” που επικοινωνούν με το REST API.

https://github.com/xoulios/festival-manager/blob/main/docs/uml/component_diagram.png

3.2 Διάγραμμα Περιβάλλοντος (Context Diagram)

Το Διάγραμμα Περιβάλλοντος αποτυπώνει τα όρια του συστήματος “Σύστημα Διαχείρισης Μουσικού Φεστιβάλ” και τη βασική αλληλεπίδρασή του με τους εξωτερικούς δρώντες (actors). Στο μοντέλο περιβάλλοντος, το σύστημα θεωρείται ως ένα ενιαίο “μαύρο κουτί”, στο οποίο καταλήγουν αιτήματα (requests) και από το οποίο επιστρέφονται αποτελέσματα (responses). Η διαφοροποίηση των ρόλων είναι κρίσιμη, καθώς οδηγεί σε διαφορετικά δικαιώματα πρόσβασης και διαφορετικές ροές εργασίας.

Visitor: Ο επισκέπτης αλληλεπιδρά με το σύστημα αποκλειστικά σε επίπεδο ανάγνωσης, μέσω λειτουργιών αναζήτησης και προβολής διαθέσιμων πληροφοριών για φεστιβάλ και παραστάσεις (“Αναζήτηση/Προβολή φεστιβάλ & παραστάσεων”). Η ροή αυτή αντιστοιχεί στην ελάχιστη δημόσια λειτουργικότητα, όπου δεν απαιτείται ταυτοποίηση.

Authenticated User: Ο αυθεντικοποιημένος χρήστης υποστηρίζεται μέσω βασικών ενεργειών πρόσβασης και διαχείρισης λογαριασμού (“Εγγραφή/Είσοδος, Προβολή προφίλ”). Στο επίπεδο περιβάλλοντος, ο χρήστης αυτός αποτελεί τη “βάση” πάνω στην οποία εξειδικεύονται οι λειτουργικοί ρόλοι (π.χ. Artist, Staff, Organizer), ανάλογα με την ανάθεση δικαιωμάτων στο πλαίσιο ενός φεστιβάλ/προγράμματος.

Artist: Ο ρόλος του καλλιτέχνη/υποβάλλοντα επικεντρώνεται στη διαχείριση περιεχομένου που αφορά παραστάσεις. Η αλληλεπίδραση που αποτυπώνεται είναι “Δημιουργία/Ενημέρωση/Υποβολή, Τελική υποβολή”, δηλαδή η παραγωγή/συντήρηση της πληροφορίας της παράστασης και η προώθησή της στο workflow για αξιολόγηση.

Staff: Ο ρόλος του προσωπικού συνδέεται άμεσα με τον έλεγχο ποιότητας/αξιολόγηση. Στο διάγραμμα εμφανίζεται ως “Αξιολόγηση (βαθμολογία/σχόλια)”, δηλαδή καταχώριση review και συμμετοχή στη διαδικασία λήψης απόφασης σύμφωνα με τους κανόνες και τις καταστάσεις του συστήματος.

Organizer: Ο οργανωτής έχει την πιο “διαχειριστική” σχέση με το σύστημα και καλύπτει ευρύτερο φάσμα λειτουργιών (“Διαχείριση φεστιβάλ, ρόλοι, states, αποφάσεις”). Στο επίπεδο περιβάλλοντος αυτό σημαίνει ότι ο organizer καθορίζει πλαίσιο λειτουργίας (π.χ. φεστιβάλ/πρόγραμμα, ανάθεση ρόλων), καθώς και κρίσιμες αποφάσεις που επηρεάζουν την τελική κατάσταση παραστάσεων ή/και τη διαμόρφωση του προγράμματος.

Συνολικά, το context diagram επιβεβαιώνει ότι ο πυρήνας του συστήματος είναι role-driven: οι ίδιοι πόροι (φεστιβάλ, παραστάσεις, αξιολογήσεις) εκτίθενται με διαφορετικό τρόπο, ανάλογα με το αν ο χρήστης είναι επισκέπτης, απλός αυθεντικοποιημένος ή μέλος της οργανωτικής/αξιολογητικής διαδικασίας.

https://github.com/xoulios/festival-manager/blob/main/docs/uml/context_diagram.png

3.3 Διαγράμματα Περιπτώσεων Χρήσης (Use Case Diagrams)

Τα Use Case Diagrams αποτυπώνουν τη λειτουργικότητα του συστήματος από την οπτική του χρήστη, δηλαδή ποιοι δρώντες (actors) αλληλεπιδρούν με το σύστημα και ποιες ενέργειες υποστηρίζονται. Στην εργασία χρησιμοποιήθηκαν δύο διαγράμματα: ένα διάγραμμα για την αυθεντικοποίηση/προφίλ και ένα διάγραμμα για τον κύριο κορμό διαχείρισης (φεστιβάλ–παραστάσεις–αξιολόγηση).

Use Case Diagram — Αυθεντικοποίηση και Βασικές ενέργειες χρήστη

Στο διάγραμμα αυθεντικοποίησης ο βασικός actor είναι ο **Authenticated User** (και έμμεσα ο Visitor που μεταβαίνει σε authenticated κατάσταση). Οι περιπτώσεις χρήσης που καλύπτονται αφορούν την είσοδο στο σύστημα και τη βασική ταυτοποίηση, καθώς και λειτουργίες που σχετίζονται με την πρόσβαση σε προσωπικές πληροφορίες (π.χ. προβολή στοιχείων/ρόλων). Η ενότητα αυτή είναι απαραίτητη διότι όλες οι κρίσιμες ενέργειες του συστήματος (δημιουργία,

ενημέρωση, αξιολόγηση) προϋποθέτουν authentication και στη συνέχεια authorization. Στην πράξη, ο στόχος του διαγράμματος είναι να ξεχωρίσει τις “γενικές” ενέργειες χρήστη από τις “εξειδικευμένες” ενέργειες ρόλων (Artist/Staff/Organizer), οι οποίες μοντελοποιούνται στο κύριο διάγραμμα.

https://github.com/xoulios/festival-manager/blob/main/docs/uml/use_case_auth.png

Use Case Diagram — Κύριες λειτουργίες διαχείρισης φεστιβάλ και παραστάσεων

Το κύριο διάγραμμα καλύπτει τον κορμό του πληροφοριακού συστήματος και οργανώνεται γύρω από τους actors:

Visitor, Artist, Staff, Organizer (με την παραδοχή ότι οι ρόλοι αυτοί ισχύουν αφού ο χρήστης είναι authenticated και έχει αντίστοιχη ανάθεση στο πλαίσιο ενός φεστιβάλ).

Οι κεντρικές περιπτώσεις χρήσης ομαδοποιούνται εννοιολογικά ως εξής:

(α) Διαχείριση Φεστιβάλ/Προγράμματος (Organizer/Staff):

Περιλαμβάνει ενέργειες δημιουργίας/ενημέρωσης δομών που ορίζουν το πλαίσιο λειτουργίας (π.χ. στοιχεία φεστιβάλ/προγράμματος, χρονικές φάσεις/καταστάσεις όπου εφαρμόζεται). Αυτό το κομμάτι είναι απαραίτητο για να επιβληθούν οι κανόνες ροής (workflow) που ακολουθούν οι παραστάσεις.

(β) Διαχείριση Παραστάσεων (Artist/Submitter, Organizer/Staff):

Περιλαμβάνει δημιουργία και ενημέρωση παράστασης και ενέργειες υποβολής (submit). Η υποβολή αποτελεί κρίσιμη περίπτωση χρήσης, καθώς μεταφέρει την παράσταση σε κατάσταση αξιολόγησης και ενεργοποιεί τους κανόνες πρόσβασης που καθορίζουν ποιος μπορεί να κάνει review/decision.

(γ) Αξιολόγηση και Απόφαση (Staff/Organizer):

Περιλαμβάνει καταχώριση αξιολόγησης (review με βαθμολογία/σχόλια) και λήψη απόφασης (approve/reject) σύμφωνα με τον κύκλο ζωής της παράστασης.

(δ) Αναζήτηση και Προβολή (Visitor/Authenticated User, role-aware):

Περιλαμβάνει αναζήτηση παραστάσεων (με λέξεις-κλειδιά/κριτήρια) και προβολή λεπτομερειών. Στο συγκεκριμένο σύστημα η προβολή είναι role-aware, δηλαδή ο ίδιος πόρος μπορεί να εμφανίζεται με διαφορετικό επίπεδο πληροφορίας ανάλογα με τον actor (π.χ. visitor vs staff/organizer).

https://github.com/xoulios/festival-manager/blob/main/docs/uml/use_case_core.png

3.4 Διαγράμματα Δραστηριοτήτων (Activity Diagrams)

Τα Διαγράμματα Δραστηριοτήτων χρησιμοποιήθηκαν για να αποτυπωθεί με σαφήνεια η δυναμική συμπεριφορά του συστήματος σε κρίσιμες ροές εργασίας. Η επιλογή activity diagrams είναι ιδιαίτερα χρήσιμη σε περιπτώσεις όπου υπάρχουν κανόνες (validations), συνθήκες (guards), εναλλακτικές διαδρομές και τερματισμοί με σφάλμα. Στη συνέχεια παρουσιάζονται και τεκμηριώνονται οι βασικές ροές που υλοποιούν τον πυρήνα του FestivalManager.

Activity Diagram — Δημιουργία Παράστασης (Create Performance)

Η ροή ξεκινά όταν ένας εξουσιοδοτημένος χρήστης (ανάλογα με την πολιτική ρόλων του συστήματος) επιλέξει την ενέργεια δημιουργίας παράστασης και καταχωρίσει τα απαιτούμενα πεδία. Αρχικά, το σύστημα πραγματοποιεί ελέγχους εγκυρότητας στα εισερχόμενα δεδομένα (π.χ. υποχρεωτικά πεδία, σωστό festival/program reference, βασικές επιχειρησιακές προϋποθέσεις). Εφόσον οι έλεγχοι αποτύχουν, η ροή τερματίζεται άμεσα με επιστροφή σφάλματος (invalid request / business rule violation). Σε περίπτωση επιτυχών ελέγχων, δημιουργείται η οντότητα Performance στην αρχική κατάσταση (τυπικά CREATED) και γίνεται αποθήκευση στη βάση. Στο σημείο αυτό, το διάγραμμα περιλαμβάνει προαιρετική λογική “αυτόματης ανάθεσης ρόλων” (auto-assign) ως σχεδιαστικό στόχο, δηλαδή την αυτόματη συσχέτιση submitter/artist στο πλαίσιο του φεστιβάλ, όπου αυτό απαιτείται από το μοντέλο. Στην τελική υλοποίηση, η ανάθεση ρόλων ακολουθεί κυρίως τον μηχανισμό role assignment στο festival, επομένως η ενέργεια auto-assign είτε απουσιάζει είτε προσεγγίζεται με διαφορετικό τρόπο (π.χ. με προκαθορισμένους ρόλους στο data seed). Η ροή ολοκληρώνεται με επιτυχή απάντηση (επιστροφή του αναγνωριστικού και βασικών στοιχείων της νέας παράστασης).

[https://github.com/xoulios/festival-manager/blob/main/docs/uml/Activity%20diagrams/Create%20Performance%20\(%2B%20auto-assign%20ARTIST_SUBMITTER%20role\).png](https://github.com/xoulios/festival-manager/blob/main/docs/uml/Activity%20diagrams/Create%20Performance%20(%2B%20auto-assign%20ARTIST_SUBMITTER%20role).png)

Activity Diagram — Αναζήτηση Παραστάσεων (Search Performances / Search View)

Η ροή αναζήτησης ξεκινά όταν ο χρήστης εισάγει κριτήρια αναζήτησης και προαιρετικές επιλογές ταξινόμησης. Στο πρώτο στάδιο, το σύστημα κανονικοποιεί τα κριτήρια (trim, διάσπαση σε tokens όπου εφαρμόζεται) και καθορίζει εάν θα γίνει αναζήτηση “χωρίς φίλτρα” (επιστροφή όλων των

προσβάσιμων αποτελεσμάτων) ή “με φίλτρα”. Η ουσία του διαγράμματος βρίσκεται στο ότι η αναζήτηση εφαρμόζει AND-σημασιολογία στα tokens: κάθε token πρέπει να ικανοποιείται (π.χ. να υπάρχει στο όνομα ή σε επιλεγμένα κειμενικά πεδία), διαφορετικά η εγγραφή απορρίπτεται από το αποτέλεσμα. Ακολουθώς εφαρμόζεται ταξινόμηση (sorting) με βάση το πεδίο που έχει επιλεγεί (ή προεπιλεγμένα κριτήρια). Στη συνέχεια, πριν την τελική επιστροφή δεδομένων, εφαρμόζεται “role-aware visibility”: το σύστημα δεν επιστρέφει σε όλους τους ρόλους το ίδιο πλήθος/είδος πληροφοριών. Για παράδειγμα, ένας visitor μπορεί να λάβει περιορισμένο view, ενώ staff/organizer ή ο handler μπορεί να δει περισσότερα πεδία, σύμφωνα με τους κανόνες πρόσβασης. Η ροή ολοκληρώνεται με επιστροφή λίστας αποτελεσμάτων. Το διάγραμμα επίσης περιλαμβάνει τερματισμούς σφάλματος για περιπτώσεις μη έγκυρων παραμέτρων ταξινόμησης ή για αποτυχία σύνδεσης/authorization όταν η συγκεκριμένη προβολή απαιτεί αυθεντικοποίηση.

[https://github.com/xoulios/festival-manager/blob/main/docs/uml/Activity%20diagrams/Search%20Performances%20\(AND%20filters%20%2B%20role%20visibility%20%2B%20sorting\).png](https://github.com/xoulios/festival-manager/blob/main/docs/uml/Activity%20diagrams/Search%20Performances%20(AND%20filters%20%2B%20role%20visibility%20%2B%20sorting).png)

Activity Diagram — Προβολή Παράστασης (Role-aware View)

Η προβολή μιας παράστασης ξεκινά με αίτημα ανά id ή μέσω εξειδικευμένης προβολής (π.χ. view endpoint/λειτουργία αναζήτησης με “πλούσιο view”). Αρχικά γίνεται ανάκτηση της οντότητας από τη βάση. Αν η παράσταση δεν βρεθεί, η ροή τερματίζεται με “not found”. Εφόσον βρεθεί, το σύστημα προσδιορίζει το επίπεδο πρόσβασης του χρήστη (visitor vs authenticated, καθώς και τυχόν ρόλοι στο συγκεκριμένο φεστιβάλ όπως staff/organizer/handler/artist). Με βάση αυτό το αποτέλεσμα, το activity diagram ακολουθεί διακλαδώσεις που οδηγούν σε διαφορετικό “projection” της ίδιας οντότητας: περιορισμένη προβολή με redaction πεδίων για visitor/απλό χρήστη, πλήρης προβολή για διοικητικούς ρόλους, ή ενδιάμεση προβολή για ειδικές περιπτώσεις (π.χ. handler). Η ροή ολοκληρώνεται με επιστροφή του κατάλληλου view DTO. Σχεδιαστικά, το συγκεκριμένο διάγραμμα είναι κρίσιμο διότι συνδέεται άμεσα με την απαίτηση να μην αποκαλύπτονται ευαίσθητα/μη δημοσιεύσιμα στοιχεία σε μη εξουσιοδοτημένους χρήστες, ακόμη και όταν αυτοί γνωρίζουν το id της παράστασης.

[https://github.com/xoulios/festival-manager/blob/main/docs/uml/Activity%20diagrams/View%20Performance%20by%20id%20\(role-aware%20redaction\).png](https://github.com/xoulios/festival-manager/blob/main/docs/uml/Activity%20diagrams/View%20Performance%20by%20id%20(role-aware%20redaction).png)

Activity Diagram — Ενημέρωση Παράστασης (Update Performance)

Η ενημέρωση ξεκινά όταν ένας χρήστης επιχειρεί να τροποποιήσει στοιχεία υπάρχουσας παράστασης. Το σύστημα ανακτά την παράσταση και ελέγχει αν υπάρχει. Στη συνέχεια εφαρμόζει κανόνα κατάστασης: η ενημέρωση επιτρέπεται μόνο όταν η παράσταση βρίσκεται σε επιτρεπτή κατάσταση (ενδεικτικά CREATED). Εάν η κατάσταση δεν είναι επιτρεπτή, η ροή τερματίζεται με business rule violation. Αν ο κανόνας κατάστασης ικανοποιείται, εκτελούνται έλεγχοι authorization (δικαιώματα ρόλου και, ως σχεδιαστικός στόχος, έλεγχος ιδιοκτησίας/συσχέτισης). Στην τελική υλοποίηση, ο περιορισμός “μόνο CREATED” αποτελεί τον βασικό φραγμό, ενώ ο έλεγχος owner μπορεί να είναι περιορισμένος ανάλογα με το πώς ορίζεται η ιδιοκτησία στο μοντέλο. Τέλος, πραγματοποιείται ενημέρωση των επιτρεπτών πεδίων, αποθήκευση στη βάση και επιστροφή του ενημερωμένου αποτελέσματος.

[https://github.com/xoulios/festival-manager/blob/main/docs/uml/Activity%20diagrams/Update%20Performance%20\(%CE%BC%CF%8C%CE%BD%CE%BF%20CREATED%20%2B%20owner\).drawio.png](https://github.com/xoulios/festival-manager/blob/main/docs/uml/Activity%20diagrams/Update%20Performance%20(%CE%BC%CF%8C%CE%BD%CE%BF%20CREATED%20%2B%20owner).drawio.png)

3.5 Διαγράμματα Ακολουθίας (Sequence Diagrams)

Τα Διαγράμματα Ακολουθίας αποτυπώνουν τη χρονική σειρά των μηνυμάτων/κλήσεων μεταξύ των επιπέδων του συστήματος. Στο FestivalManager, η τυπική ροή είναι: Client (Frontend ή Insomnia) → Spring Security (HTTP Basic) → Controller → Service (επιχειρησιακή λογική/κανόνες) → Repository → Βάση δεδομένων, και στη συνέχεια επιστροφή απάντησης προς τον client. Οι ακολουθίες που επιλέχθηκαν καλύπτουν τις πιο κρίσιμες λειτουργίες όπου υπάρχουν έλεγχοι ρόλων, έλεγχοι κατάστασης (festival/performance state) και πιθανές αποτυχίες με BusinessException.

SD — Search Performances View (GET /api/performances/search-view)

Η ακολουθία ξεκινά από αίτημα προβολής λίστας παραστάσεων με “view” DTO, ώστε τα αποτελέσματα να είναι role-aware. Επειδή το endpoint είναι GET, επιτρέπεται και χωρίς authentication· όταν δεν υπάρχει authenticated principal, ο χρήστης θεωρείται visitor. Στο controller, το αίτημα δρομολογείται στη μέθοδο searchView, η οποία περνά στο service το festivalId, το userId (ή null όταν δεν υπάρχει principal) και το προαιρετικό query string. Στη συνέχεια, το service

εκτελεί αρχικά αναζήτηση στη βάση μέσω `PerformanceRepository.findByFestivalId(festivalId)` και εφαρμόζει επεξεργασία `tokens` (AND-σημασιολογία) και ταξινόμηση (`genre`, έπειτα `name`). Ακολουθώντας, για κάθε `Performance` γίνεται μετασχηματισμός σε `PerformanceViewDto` μέσω της `toViewDto(p, userId)`. Εκεί πραγματοποιούνται έλεγχοι ρόλων, με κλήσεις προς `UserFestivalRoleRepository` (π.χ. `STAFF/PROGRAMMER/ORGANIZER/ARTIST`) και εφαρμόζεται ο κανόνας ορατότητας: για `visitor` επιστρέφονται μόνο παραστάσεις που βρίσκονται σε κατάσταση `SCHEDULED`, ενώ για `staff` επιτρέπεται προβολή μόνο όταν είναι ο “assigned handler” ή έχει αυξημένα δικαιώματα. Σε περιπτώσεις `programmer/handler` υπάρχει επιπλέον ανάκτηση του τελευταίου `review` μέσω `ReviewRepository.findTopByPerformanceIdOrderByDesc`, ώστε να εμφανίζονται συνοπτικά `score/comments`. Η ακολουθία ολοκληρώνεται με επιστροφή λίστας DTO προς τον `client`. Το κρίσιμο σημείο εδώ είναι ότι η “ασφάλεια προβολής” υλοποιείται μέσα στο `mapping` (`filtering + redaction`), ώστε το αποτέλεσμα να είναι συμβατό με τους κανόνες ρόλων ακόμη και σε δημόσια GET κλήση.

[https://github.com/xoulios/festival-manager/blob/main/docs/uml/Sequence%20Diagrams/Search%20Performance%20View%20\(GET%20_api_performances_search-view\).png](https://github.com/xoulios/festival-manager/blob/main/docs/uml/Sequence%20Diagrams/Search%20Performance%20View%20(GET%20_api_performances_search-view).png)

SD — Update Performance (PUT /api/performances/{id})

Η ακολουθία ενημέρωσης ξεκινά με PUT αίτημα και περνά υποχρεωτικά από authentication (non-GET). Το controller καλεί `updatePerformance(id, updated)` στο service. Στο service, πρώτα γίνεται ανάκτηση της παράστασης από `PerformanceRepository.findById(id)`. Αν δεν βρεθεί, παράγεται `BusinessRuleException` (“Performance not found”). Αν βρεθεί, ακολουθεί έλεγχος κατάστασης: επιτρέπεται ενημέρωση μόνο σε `CREATED`. Αν η κατάσταση δεν είναι `CREATED`, παράγεται `BusinessRuleException` (“Performance can be updated only in CREATED state”). Σε επιτυχία ελέγχων, ενημερώνονται τα επιτρεπτά πεδία (`name/description/genre/duration` κ.λπ.) και εκτελείται `PerformanceRepository.save(existing)`. Η ακολουθία ολοκληρώνεται με επιστροφή της ενημερωμένης οντότητας.

[https://github.com/xoulios/festival-manager/blob/main/docs/uml/Sequence%20Diagrams/Update%20Performance%20\(PUT%20_api_performances_%7Bid%7D\).png](https://github.com/xoulios/festival-manager/blob/main/docs/uml/Sequence%20Diagrams/Update%20Performance%20(PUT%20_api_performances_%7Bid%7D).png)

SD — Create Performance (POST /api/performances?festivalId=...)

Η ακολουθία δημιουργίας ξεκινά με POST αίτημα (άρα απαιτεί authentication). Το controller λαμβάνει `festivalId`, `principal.getId()` και το `request body` (`Performance`)

και καλεί `createPerformance(festivalId, artistId, performance)`. Στο service εκτελούνται δύο βασικές ανακτήσεις: `FestivalRepository.findById(festivalId)` και `UserRepository.findById(artistId)`. Αν κάποιο δεν βρεθεί, παράγεται `BusinessRuleException` (π.χ. “Festival not found”, “Artist not found”). Έπειτα ορίζονται οι συσχετίσεις (`performance.setFestival`, `performance.setArtist`) και, αν δεν έχει δοθεί state, τίθεται προεπιλεγμένα `CREATED`. Τέλος, γίνεται `PerformanceRepository.save(performance)` και επιστρέφεται HTTP 201 με το αποτέλεσμα.

[https://github.com/xoullos/festival-manager/blob/main/docs/uml/Sequence%20Diagrams/Create%20Performance%20\(POST%20api_performances\).png](https://github.com/xoullos/festival-manager/blob/main/docs/uml/Sequence%20Diagrams/Create%20Performance%20(POST%20api_performances).png)

SD — View Performance by id (GET /api/performances/{id}/view)

Η ακολουθία προβολής ανά id ξεκινά με GET αίτημα σε endpoint “view”. Ο controller καλεί το service, το οποίο:

1. Ανακτά την οντότητα Performance από τη βάση.
2. Προσδιορίζει το επίπεδο πρόσβασης (visitor ή χρήστης με ρόλο στο festival).
3. Δημιουργεί και επιστρέφει το κατάλληλο “view DTO” με **redaction** (περιορισμό/απόκρυψη πεδίων) όπου απαιτείται.

Έτσι, η ίδια παράσταση μπορεί να προβάλλεται με διαφορετικό περιεχόμενο, με βάση τα δικαιώματα χρήστη, χωρίς να απαιτείται ξεχωριστή οντότητα στη βάση. Η διαφοροποίηση υλοποιείται στο επίπεδο service/DTO mapping.

[https://github.com/xoullos/festival-manager/blob/main/docs/uml/Sequence%20Diagrams/View%20Performance%20by%20id%20\(GET%20api_performances_%7Bid%7D_view\).drawio.png](https://github.com/xoullos/festival-manager/blob/main/docs/uml/Sequence%20Diagrams/View%20Performance%20by%20id%20(GET%20api_performances_%7Bid%7D_view).drawio.png)

3.6 Διάγραμμα Οντοτήτων–Συσχετίσεων (Entity–Relationship Diagram)

Το ER διάγραμμα αποτυπώνει το μοντέλο δεδομένων του FestivalManager όπως αυτό υλοποιείται στη βάση (μέσω JPA entities). Ο πυρήνας της εφαρμογής οργανώνεται γύρω από τις οντότητες **Festival**, **Performance** και **Review**, ενώ η διαχείριση πρόσβασης/δικαιωμάτων εκφράζεται μέσω των οντοτήτων **User**, **Role** και του συσχετιστικού πίνακα **UserFestivalRole** (ρόλος χρήστη ανά φεστιβάλ).

Οντότητες και βασικά γνωρίσματα

Festival

Αποθηκεύει τα βασικά στοιχεία του φεστιβάλ: title (μοναδικό), description, startDate, endDate, καθώς και την state (FestivalState). Κάθε festival αποτελεί “container” για performances.

Performance

Αποθηκεύει στοιχεία παράστασης: name, description, genre, durationMinutes και επιπλέον πεδία λειτουργικού workflow (π.χ. submittedAt, reviewedAt, finalSubmittedAt, scheduledSlot, finalSetlist, rejectionReason, και state ως PerformanceState). Η Performance συνδέεται υποχρεωτικά με festival και με artist, ενώ προαιρετικά με handler.

Review

Αποθηκεύει αξιολόγηση: score, comments και συνδέεται υποχρεωτικά με performance και reviewer (user).

User

Αποθηκεύει στοιχεία χρήστη: username (μοναδικό), email (μοναδικό), password (hash), enabled. Χρησιμοποιείται τόσο ως artist/submitter όσο και ως reviewer/handler, ανάλογα με τη συσχέτιση.

Role

Αποθηκεύει τους διαθέσιμους ρόλους (π.χ. ARTIST, STAFF, ORGANIZER, PROGRAMMER) ως name (μοναδικό).

UserFestivalRole (συσχετιστική οντότητα)

Υλοποιεί την ανάθεση ρόλου χρήστη σε συγκεκριμένο festival. Περιλαμβάνει ξένα κλειδιά προς User, Festival, Role και σύνθετο κλειδί (embedded id). Επιπλέον επιβάλλεται περιορισμός ώστε για κάθε ζεύγος (user, festival) να αντιστοιχεί ουσιαστικά μία ανάθεση ρόλου.

Σχέσεις και πληθικότητες

Festival – Performance (1:N)

Ένα festival μπορεί να έχει πολλές performances. Κάθε performance ανήκει σε ακριβώς ένα festival (festival_id υποχρεωτικό).

User – Performance ως Artist (1:N)

Ένας user μπορεί να είναι artist σε πολλές performances. Κάθε performance έχει ακριβώς έναν artist (artist_id υποχρεωτικό).

User – Performance ως Handler (0/1:N)

Μία performance μπορεί να έχει προαιρετικά έναν handler (handler_id nullable). Ένας user μπορεί να είναι handler σε πολλές performances.

Performance – Review (1:N)

Μία performance μπορεί να έχει πολλές reviews. Κάθε review ανήκει σε ακριβώς μία performance (performance_id υποχρεωτικό).

User – Review ως Reviewer (1:N)

Ένας user μπορεί να έχει κάνει πολλές reviews. Κάθε review ανήκει σε ακριβώς έναν reviewer (reviewer_id υποχρεωτικό).

User – Festival – Role (N:M μέσω UserFestivalRole)

Η ανάθεση ρόλων δεν αποθηκεύεται ως απλό πεδίο στον user. Αντίθετα, υλοποιείται ως ξεχωριστή συσχετιστική οντότητα: ένας user μπορεί να συμμετέχει σε πολλά festivals και κάθε festival να έχει πολλούς users με ρόλους. Η οντότητα UserFestivalRole συνδέει και τα τρία (user, festival, role) και επιτρέπει role-based authorization σε επίπεδο festival.

Περιορισμοί ακεραιότητας και σχεδιαστικές επιλογές

Στο μοντέλο εφαρμόζονται περιορισμοί μοναδικότητας σε επίπεδο βάσης, όπως Festival.title, User.username, User.email, Role.name. Επίσης, η ύπαρξη των ξένων κλειδιών festival_id, artist_id, performance_id, reviewer_id εξασφαλίζει ότι δεν μπορεί να υπάρξει performance χωρίς festival/artist και review χωρίς performance/reviewer. Τέλος, οι καταστάσεις (FestivalState, PerformanceState) αποθηκεύονται ως enums (string representation) και χρησιμοποιούνται ως βάση για τους κανόνες workflow (π.χ. πότε επιτρέπεται submit, review, decision, scheduling) που αναλύονται στα Activity και Sequence Diagrams και αντιστοιχούν σε αντίστοιχους ελέγχους στην επιχειρησιακή λογική.

https://github.com/xoulios/festival-manager/blob/main/docs/uml/er_diagram.drawio.png

Υλοποίηση συστήματος/εφαρμογής

4.1 GitHub Repository και τεκμηρίωση λειτουργίας API endpoints

Η υλοποίηση του συστήματος αναπτύχθηκε σε αποθετήριο GitHub, ώστε να είναι εφικτή η ιχνηλασιμότητα της ανάπτυξης (ιστορικό commits), η οργάνωση των παραδοτέων και η αναπαραγωγή της εκτέλεσης από τρίτους. Το repository περιλαμβάνει τον κώδικα του backend και του frontend, καθώς και συνοδευτικά αρχεία που απαιτούνται για την εκτέλεση του έργου.

Σύνδεσμος αποθετηρίου: [<https://github.com/xoullos/festival-manager>]

Κλάδος παράδοσης: [main/dev]

Οδηγίες εκτέλεσης: [README]

Η επιβεβαίωση της ορθής λειτουργίας των API endpoints πραγματοποιήθηκε με πρακτικές κλήσεις HTTP. Συγκεκριμένα, τα endpoints ελέγχθηκαν με τον ακόλουθο τρόπο: (α) μέσω της ενσωματωμένης τεκμηρίωσης API (backend), (β) με άμεσες κλήσεις από γραμμή εντολών (curl) ή/και (γ) μέσω των πραγματικών ροών του frontend που καταναλώνουν τα ίδια endpoints. Με αυτόν τον έλεγχο επιβεβαιώθηκαν τόσο τα “επιτυχημένα” σενάρια (200/201/204), όσο και τα σενάρια αποτυχίας που δείχνουν ότι οι έλεγχοι ασφάλειας και εγκυρότητας λειτουργούν (401 χωρίς αυθεντικοποίηση, 403 με ανεπαρκή δικαιώματα, 400/422 σε μη έγκυρα δεδομένα).

4.2 Υλοποίηση και σχολιασμός

Η εφαρμογή υλοποιήθηκε με σαφή διαχωρισμό επιπέδων, ώστε να είναι ευκολότερη η συντήρηση και ο έλεγχος. Το backend υλοποιεί την επιχειρησιακή λογική, τους κανόνες και την πρόσβαση στη βάση, ενώ το frontend υλοποιεί την παρουσίαση και τις ροές χρήσης. Η επικοινωνία πραγματοποιείται μέσω REST API με ανταλλαγή JSON.

Δομή έργου

Το έργο διαχωρίζεται σε δύο κύρια υποσυστήματα.

Backend (Spring Boot): Στην πλευρά του backend, η υλοποίηση οργανώνεται σε επίπεδο controllers (REST endpoints), services (επιχειρησιακοί κανόνες), repositories (πρόσβαση δεδομένων) και entities/DTOs (μοντέλο και μεταφορά δεδομένων). Ο διαχωρισμός αυτός διευκολύνει τη δοκιμή (unit tests στο service layer).

Frontend (React/TypeScript): Στην πλευρά του frontend, υλοποιούνται σελίδες (pages) για τις βασικές λειτουργίες, επαναχρησιμοποιήσιμα components για UI, καθώς και μηχανισμός κατάστασης/συνεδρίας (π.χ. context) για αυθεντικοποίηση και διαχείριση token.

Backend: βασικές κλάσεις/αρχεία και βασικές ροές

Οι controllers εκθέτουν τα endpoints και αναλαμβάνουν τη λήψη/επιστροφή δεδομένων σε JSON, ενώ οι services υλοποιούν τους κανόνες του συστήματος. Σε αυτό το επίπεδο εφαρμόζονται έλεγχοι όπως εγκυρότητα δεδομένων, δικαιώματα χρήστη, καθώς και περιορισμοί ροής. Οι repositories χρησιμοποιούνται για την πρόσβαση στη βάση (CRUD), ενώ όπου απαιτείται χρησιμοποιούνται ειδικά queries για αναζητήσεις με φίλτρα και ταξινόμηση. Η δομή αυτή κρατάει καθαρό το endpoint layer και συγκεντρώνει τη λογική σε επαναχρησιμοποιήσιμες μεθόδους.

Frontend: βασικές κλάσεις/αρχεία και βασικές ροές

Το frontend υλοποιεί ροές χρήσης που αντιστοιχούν άμεσα στις λειτουργίες του backend, όπως προβολή λιστών, προβολή λεπτομερειών, δημιουργία/ενημέρωση οντοτήτων και εκτέλεση ενεργειών που μεταβάλλουν κατάσταση. Η αυθεντικοποίηση πραγματοποιείται στο login και το token (όπου υπάρχει) χρησιμοποιείται στις επόμενες κλήσεις API. Σημαντικό στοιχείο της υλοποίησης είναι ότι το UI προσαρμόζεται ανά ρόλο, τόσο στην εμφάνιση δεδομένων όσο και στις διαθέσιμες ενέργειες. Έτσι, οι κανόνες πρόσβασης εφαρμόζονται πρακτικά σε δύο επίπεδα: στο backend ως τελική εγγύηση ασφάλειας, και στο frontend ως καθοδήγηση χρήστη και καλύτερη εμπειρία χρήσης.

Βάση σε local/dev περιβάλλον

Κατά την ανάπτυξη χρησιμοποιήθηκε local/dev βάση H2, ώστε να είναι δυνατή η γρήγορη εκτέλεση και δοκιμή χωρίς ανάγκη εξωτερικού DB server. Η H2 είναι επαρκής για την επιβεβαίωση της λειτουργίας των endpoints, καθώς όλες οι CRUD λειτουργίες και οι αναζητήσεις εκτελούνται σε πραγματική βάση και μπορούν να επαληθευτούν με ακολουθία “write → read” (π.χ. POST/PUT και στη συνέχεια GET για επιβεβαίωση αποθήκευσης).

4.3 Τεκμηρίωση δοκιμών

Η διαδικασία δοκιμών στοχεύει στο να διασφαλιστεί ότι οι βασικές λειτουργίες του συστήματος εκτελούνται ορθά, ότι τηρούνται οι κανόνες πρόσβασης και εγκυρότητας και ότι αποφεύγονται σφάλματα παλινδρόμησης μετά από αλλαγές.

Unit tests και κάλυψη

Υλοποιήθηκαν unit tests σε κρίσιμα σημεία του service layer, όπου ελέγχονται οι βασικοί επιχειρησιακοί κανόνες. Στα tests επιβεβαιώνονται τόσο θετικά σενάρια (σωστή εκτέλεση), όσο και αρνητικά (σωστό exception/status όταν κάτι παραβιάζει κανόνες). Όπου ήταν απαραίτητο χρησιμοποιήθηκαν mocks για repositories/εξαρτήσεις, ώστε να απομονώνεται η υπό δοκιμή λογική και να παραμένουν τα tests γρήγορα και αξιόπιστα.

```

File Edit View Search Terminal Help
xoulios@Xoulios: ~/festival-manager/backend/festival-manager
2026-01-13T21:17:27.165402:00 INFO 4979 --- [main] o.s.b.f.w.SpringBootMockServletContext : Initializing Spring TestDispatcherServlet
2026-01-13T21:17:27.167402:00 INFO 4979 --- [main] o.s.t.web.servlet.TestDispatcherServlet : Initializing Servlet
2026-01-13T21:17:27.168402:00 INFO 4979 --- [main] o.s.t.web.servlet.TestDispatcherServlet : Completed initialization in 1 ms
2026-01-13T21:17:27.177402:00 INFO 4979 --- [main] g.u.f.controller.FestivalControllerTest : Started FestivalControllerTest in 0.56 seconds (process running for 7.279)
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.624 s -- in gr.uoi.festivalmanager.controller.FestivalControllerTest
[INFO] Running gr.uoi.festivalmanager.FestivalServiceImpTest
2026-01-13T21:17:27.126402:00 INFO 4979 --- [main] t.c.s.AnnotationConfigContextLoaderUtils : Could not detect default configuration classes for test class [gr.uoi.festivalmanager.FestivalServiceImpTest]: FestivalServiceImpTest does not declare any static, non-private, non-final, nested classes annotated with @Configuration.
2026-01-13T21:17:27.128402:00 INFO 4979 --- [main] b.t.t.SpringBootTestContextBootstrapper : Found @SpringBootConfiguration gr.uoi.festivalmanager.FestivalManagerApplication for test class gr.uoi.festivalmanager.FestivalServiceImpTest

:: Spring Boot ::
(v3.4.1)

2026-01-13T21:17:27.165402:00 INFO 4979 --- [main] g.u.f.FestivalServiceImpTest : Starting FestivalServiceImpTest using Java 21.0.9 with PID 4979 (started by xoulios in /home/xoulios/festival-manager/backend/festival-manager)
2026-01-13T21:17:27.165402:00 INFO 4979 --- [main] g.u.f.FestivalServiceImpTest : The following 1 profile is active: "test"
2026-01-13T21:17:27.331402:00 INFO 4979 --- [main] s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-01-13T21:17:27.337402:00 INFO 4979 --- [main] s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 24 ms. Found 6 JPA repository interfaces.
2026-01-13T21:17:27.456402:00 INFO 4979 --- [main] o.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing PersistenceUnitInfo [name: default]
2026-01-13T21:17:27.458402:00 INFO 4979 --- [main] o.h.c.internal.RegionFactoryInitiator : HHH000026: Second-level cache disabled
2026-01-13T21:17:27.471402:00 INFO 4979 --- [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup; ignoring JPA class transformer
2026-01-13T21:17:27.472402:00 INFO 4979 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-3 - Starting...
2026-01-13T21:17:27.473402:00 INFO 4979 --- [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-3 - Added connection conn0: url=jdbc:h2:mem:testdb user=SA
2026-01-13T21:17:27.474402:00 INFO 4979 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-3 - Start completed.
2026-01-13T21:17:27.474402:00 INFO 4979 --- [main] org.hibernate.orm.connections.pooling : HHH0001005: Database info:
Database version: 2.0.232
Database driver: undefined/unknown
Database url: undefined/unknown
Database user: undefined/unknown
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Maximum pool size: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-01-13T21:17:27.633402:00 INFO 4979 --- [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH0000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-01-13T21:17:27.656402:00 INFO 4979 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-01-13T21:17:27.656402:00 WARN 4979 --- [main] jpaBaseConfigurationJpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-01-13T21:17:27.888402:00 INFO 4979 --- [main] r.InitializeUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsServiceImpl bean with name customUserDetailsService
2026-01-13T21:17:27.892402:00 INFO 4979 --- [main] g.u.f.FestivalServiceImpTest : Started FestivalServiceImpTest in 0.861 seconds (process running for 8.195)
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.036 s -- in gr.uoi.festivalmanager.FestivalServiceImpTest
[INFO] Results:
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 12.147 s
[INFO] Finished at: 2026-01-13T21:17:27.28402:00
[INFO]
xoulios@Xoulios: ~/festival-manager/backend/festival-manager$
  
```

Εκτελέστηκαν αυτοματοποιημένες δοκιμές (JUnit μέσω Maven Surefire) με την εντολή `./mvnw test`. Οι δοκιμές ολοκληρώθηκαν επιτυχώς (BUILD SUCCESS) χωρίς αποτυχίες ή σφάλματα. Συνολικά εκτελέστηκαν 9 tests (Failures: 0, Errors: 0, Skipped: 0). Κατά την εκτέλεση χρησιμοποιήθηκε το test profile και in-memory βάση H2 (`jdbc:h2:mem:testdb`), ώστε οι δοκιμές να είναι αναπαραγώγιμες χωρίς εξωτερική εγκατάσταση DB.

Προϋποθέσεις εκτέλεσης και εντολές

Για την εκτέλεση των δοκιμών απαιτείται εγκατεστημένο JDK και το build εργαλείο του έργου. Η εκτέλεση γίνεται από τη ρίζα του backend με την εντολή:

`./mvnw test`

Σε local/dev περιβάλλον, η χρήση H2 επιτρέπει την εκτέλεση των tests χωρίς εξωτερική βάση.

Σύνδεση automated tests και επιβεβαίωσης API

Τα unit tests συμπληρώνουν τον χειροκίνητο έλεγχο των endpoints. Ο χειροκίνητος έλεγχος (Swagger/curl/Frontend) δείχνει την πραγματική συμπεριφορά του API σε επίπεδο HTTP, ενώ τα unit tests επιβεβαιώνουν συστηματικά ότι οι βασικοί κανόνες συνεχίζουν να ισχύουν μετά από αλλαγές στον κώδικα. Με τον συνδυασμό αυτών, τεκμηριώνεται ότι το σύστημα είναι λειτουργικό και σταθερό στα βασικά σενάρια.

Συμπεράσματα

Η παρούσα εργασία είχε ως στόχο την ανάλυση, σχεδίαση και υλοποίηση ενός ολοκληρωμένου πληροφοριακού συστήματος για τη διαχείριση λειτουργιών σχετικών με φεστιβάλ. Μέσα από τα στάδια της εργασίας εφαρμόστηκαν πρακτικές τεχνολογίας λογισμικού, από τη μετατροπή απαιτήσεων σε τεχνικές προδιαγραφές, έως την υλοποίηση και την τεκμηρίωση. Σε επίπεδο υλοποίησης, αναπτύχθηκε backend με Spring Boot και REST API, με διαχωρισμό αρμοδιοτήτων σε controllers, services και repositories, ώστε η επιχειρησιακή λογική να παραμένει συγκεντρωμένη και ελέγξιμη. Ιδιαίτερη έμφαση δόθηκε στους κανόνες εγκυρότητας (validations), στη σωστή διαχείριση σφαλμάτων και στους ελέγχους αυθεντικοποίησης/εξουσιοδότησης, ώστε οι λειτουργίες να εκτελούνται μόνο από χρήστες με τα αντίστοιχα δικαιώματα. Παράλληλα, το frontend υλοποιήθηκε με React, παρέχοντας τις βασικές ροές χρήσης και μια πιο άμεση αλληλεπίδραση με το σύστημα, αξιοποιώντας τα endpoints του backend. Για τις ανάγκες ανάπτυξης και δοκιμών χρησιμοποιήθηκε local/test περιβάλλον με H2 (in-memory), το οποίο επέτρεψε γρήγορη εκτέλεση και επαναληψιμότητα χωρίς εξωτερικές εξαρτήσεις. Η σταθερότητα της υλοποίησης υποστηρίχθηκε από αυτοματοποιημένες δοκιμές, οι οποίες εκτελέστηκαν επιτυχώς (Build Success, χωρίς failures ή errors). Σε συνδυασμό με τους χειροκίνητους ελέγχους των ρόλων, τεκμηριώνεται ότι οι κύριες λειτουργίες του συστήματος είναι λειτουργικές και σύμφωνες με τον τρόπο που σχεδιάστηκαν. Κατά τη διάρκεια της εργασίας προέκυψαν πρακτικά συμπεράσματα σχετικά με την αξία της σωστής οργάνωσης κώδικα και της τήρησης των επιπέδων αρχιτεκτονικής, αλλά και με τη σημασία των ρόλων/δικαιωμάτων σε πραγματικά συστήματα, όπου η

ίδια πληροφορία δεν είναι πάντα ορατή ή επεξεργάσιμη από όλους. Επιπλέον, φάνηκε ότι η τεκμηρίωση (διαγράμματα, περιγραφή ροών, δοκιμές) δεν λειτουργεί μόνο ως “παράδοση”, αλλά βοηθά ουσιαστικά στη συντήρηση και στην κατανόηση της εφαρμογής.

Ως μελλοντικές βελτιώσεις μπορούν να αναφερθούν η ενίσχυση της κάλυψης δοκιμών με περισσότερα integration tests σε κρίσιμες ροές, η περαιτέρω σκλήρυνση του security (π.χ. πιο λεπτομερής πολιτική ρόλων και audit logs), καθώς και η βελτίωση της εμπειρίας χρήστη στο frontend με περισσότερη καθοδήγηση/μηνύματα σε σενάρια αποτυχίας. Τέλος, παρότι η H2 εξυπηρέτησε αποτελεσματικά την ανάπτυξη και την επιβεβαίωση λειτουργίας, μια πλήρης “παραγωγική” εγκατάσταση μπορεί να αξιοποιήσει εξωτερική βάση (π.χ. MariaDB/MySQL), ώστε να προσομοιώνεται ακόμη πιο ρεαλιστικά η λειτουργία ενός τελικού συστήματος.

Συνολικά, η εργασία οδήγησε σε ένα λειτουργικό σύστημα με σαφή αρχιτεκτονική, τεκμηριωμένες ροές και επαληθευμένη λειτουργία μέσω δοκιμών, επιτυγχάνοντας τους βασικούς στόχους που τέθηκαν από την εκκώνηση.

Πηγές

- <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
- <https://docs.spring.io/spring-security/reference/>
- <https://docs.spring.io/spring-data/jpa/reference/>
- <https://maven.apache.org/guides/>
- <https://junit.org/junit5/docs/current/user-guide/>
- <https://jwt.io/introduction>
- <http://www.h2database.com/html/quickstart.html>
- <https://mariadb.com/docs>
- <https://git-scm.com/book/en/v2>
- https://roy.gbiv.com/pubs/dissertation/rest_arch_style.htm
- <https://martinfowler.com/articles/practical-test-pyramid.html>
- <https://beanvalidation.org>
- <https://www.rfc-editor.org/rfc/rfc9457.html>

- <https://learning.postman.com/docs/introduction/overview/>
- <https://learn.microsoft.com/azure/architecture/best-practices/api-design>